

Dynamic Programming

From Intuition to Pattern Recognition

A derivation-first guide to recognizing, classifying, and solving Dynamic Programming problems — built from a complete learning session covering Prefix, Interval, Two-Sequence, Grid, Knapsack, Tree, and Bitmask DP.

Prefix DP

Interval DP

Two-Sequence DP

Grid DP

Knapsack DP

Tree DP

Bitmask DP

“Do not memorize problem names. Recognize the shape of the subproblem.”

Contents

1. What is Dynamic Programming?	4
1.1 Overlapping Subproblems	4
1.2 Optimal Substructure	4
1.3 The Most Important Word in DP: STATE	4
1.4 DP Is NOT Simply “Use an Array Called dp”	5
2. The Two Properties of DP	5
3. DP vs. Greedy	5
4. How to Identify a DP Problem	6
Clue 1 — The problem asks for an optimum, count, or possibility	6
Clue 2 — There are choices at each step	6
Clue 3 — Choices lead to smaller versions of the same problem	6
Clue 4 — Smaller problems overlap	7
Recognition Checklist	7
Decision Tree	7
5. The Universal DP Problem-Solving Procedure	7
Step 1 — Identify the Pattern	8
Step 2 — Define the State	8
Step 3 — Identify the Choices / Last Decision	8
Step 4 — Write the Recurrence	8
Step 5 — Define Base Cases	9
Step 6 — Computation Order	9
Step 7 — Implement	9
Step 8 — Final Answer	9
6. DP Classification by State Shape	9
Critical Distinction: When <code>dp[i][j]</code> Appears	10
7. Prefix DP	10
7.1 Climbing Stairs	11
7.2 House Robber	12
7.3 Decode Ways	13
7.4 Coin Change	15
7.5 Longest Increasing Subsequence (LIS)	16
7.6 Word Break	18
8. Two-Sequence DP	19
8.1 Longest Common Subsequence (LCS)	20
8.2 Edit Distance	22
8.3 Distinct Subsequences	24
9. Interval DP	26
Prefix DP vs. Interval DP	27

10. Shrinking Interval DP	27
Longest Palindromic Subsequence (LPS)	27
11. Partition DP	29
12. Matrix Chain Multiplication (MCM)	30
The Problem (Understanding First)	30
DP Derivation	30
13. Grid DP	33
13.1 Minimum Path Sum	33
13.2 Unique Paths	34
13.3 Maximal Square	35
14. Knapsack DP	38
14.1 Partition Equal Subset Sum (0/1 Knapsack)	38
14.2 1D Optimization	40
15. Tree DP	42
15.1 House Robber III (Full Derivation)	42
16. Bitmask DP	45
The Problem First: Travelling Salesman Problem (TSP)	45
Why Ordinary DP Fails	46
The Bitmask as a Subset	46
TSP DP State	46
Knapsack vs. Bitmask	49
17. Memoization vs. Tabulation	49
Memoization (Top-Down)	49
Tabulation (Bottom-Up)	49
When to Use Each	50
18. Reusable DP Templates	50
Prefix DP — Memoization	50
Prefix DP — Tabulation	50
Two-Sequence DP — Memoization	51
Two-Sequence DP — Tabulation	51
Shrinking Interval DP — Memoization	51
Shrinking Interval DP — Tabulation	52
Partition Interval DP — Memoization	52
Partition Interval DP — Tabulation	53
Tree DP Template	53
19. Important Distinctions	54
Two-Sequence DP vs. Interval DP	54
Shrinking Interval vs. Partition Interval	54
0/1 Knapsack vs. Unbounded Knapsack	55
20. Recognition Cheat Sheet	55

Recognition Table	55
21. The DP Thinking Procedure — One Page	56
22. Common Mistakes	56

1. What is Dynamic Programming?

The simplest useful definition:

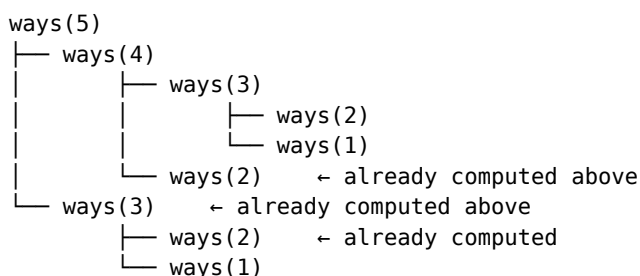
Dynamic Programming is a technique for solving a problem by breaking it into smaller subproblems, solving each subproblem once, and reusing those results.

Three ideas are hidden inside that definition.

1.1 Overlapping Subproblems

Consider: “How many ways can I climb 5 stairs if I can take 1 or 2 steps at a time?”

To reach stair 5, you must come from stair 4 (1-step) or stair 3 (2-step):



ways(3) and ways(2) appear **multiple times**. These are overlapping subproblems.

Without DP:

```

ways(3) → calculate
ways(3) → calculate again
ways(3) → calculate again ← wasted work
  
```

With DP:

```

ways(3) = 3 → calculate ONCE, store
ways(3) → reuse stored result
ways(3) → reuse stored result
  
```

1.2 Optimal Substructure

An optimal solution to the whole problem can be constructed from optimal solutions to its smaller subproblems.

Example — Minimum Path Sum:

```

best path to (r,c)
    ↓
best path to (r-1,c)
    OR
best path to (r,c-1)
  
```

If the best path to (r, c) passes through $(r-1, c)$, then the sub-path reaching $(r-1, c)$ must itself be optimal. You don't need a non-optimal path to a subproblem to produce the optimal overall answer.

1.3 The Most Important Word in DP: STATE

A **state** represents one particular smaller version of the original problem.

`dp[i]` → answer for the first i elements
`dp[i][j]` → answer for first i elements of A and first j of B
`dp[l][r]` → answer for the interval from l to r

The shape of the state is what helps you classify the DP problem.

This is the single most important idea in this material.

1.4 DP Is NOT Simply “Use an Array Called dp”

The thinking process is:

```

Problem
↓
Can I break it into smaller subproblems?
↓
Do smaller subproblems repeat?
↓
Can their answers be reused?
↓
DP may be appropriate
↓
What makes one subproblem different from another?
↓
Define the STATE
↓
State shape gives PATTERN
  
```

2. The Two Properties of DP

Both must be present for DP to work well:

Property	What it means
Overlapping Subproblems	The same smaller problems are solved repeatedly in a naive recursion
Optimal Substructure	The optimal answer to the big problem is built from optimal answers to smaller problems

DP eligibility

```

|
├─ Overlapping Subproblems → same subproblems repeat
├─ Optimal Substructure → optimal big = f(optimal small)
  
```

3. DP vs. Greedy

Both require **Optimal Substructure**. The difference is in the other property:

Technique	Required Properties
DP	Overlapping Subproblems + Optimal Substructure
Greedy	Greedy Choice Property + Optimal Substructure

Greedy Choice Property: You can make the best local choice *right now*, and that choice is safe — it won't prevent you from reaching a global optimum. Once chosen, you never look back.

Mental difference:

DP:

"What is the optimal answer for this subproblem?"
(explore all choices, pick the best)

Greedy:

"Can I make this choice now and never regret it?"
(make one choice, move on)

A greedy algorithm commits. DP evaluates. If making a local choice now could require revisiting that choice later, greedy won't work — DP might.

4. How to Identify a DP Problem

There is no single magic sentence. You look for a combination of clues.

Clue 1 — The problem asks for an optimum, count, or possibility

- Maximum / minimum value, length, cost, number of operations
- Number of ways / distinct paths / valid decodings
- Is it possible to reach a target / form a sum?

These are strong signals, but not sufficient alone. "Find the maximum element in an array" is optimization but not DP.

Clue 2 — There are choices at each step

- Rob or don't rob a house
- Take or don't take an item
- Decode 1 digit or 2 digits
- Split here or split there

Ask yourself: "What choices do I have?"

Clue 3 — Choices lead to smaller versions of the same problem

Original problem

↓

Choice

↙

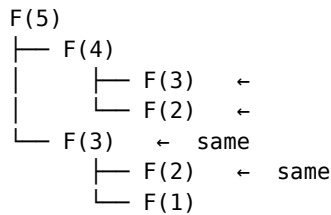
smaller
problem

↘

smaller
problem

Both smaller problems are the *same type* as the original.

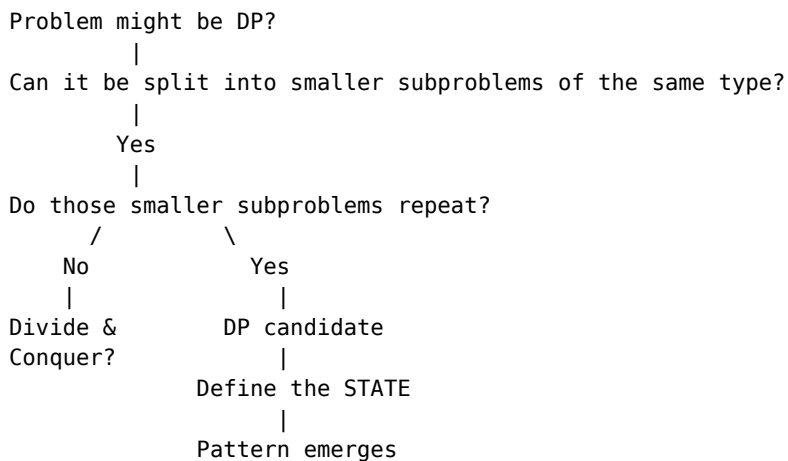
Clue 4 — Smaller problems overlap



Recognition Checklist

1. Is there a smaller version of the same problem?
2. Do the same smaller problems appear repeatedly?
3. Can I describe the problem using a small state?
4. Does the answer depend on previous decisions?
5. Can I express the answer in terms of smaller states?

Decision Tree



This is a recognition aid, not a mathematical proof. Use it to guide your thinking.

The ultimate goal: Look at a new problem and ask:

“What makes one subproblem different from another?” → That determines the state → The state shape determines the pattern.

5. The Universal DP Problem-Solving Procedure

Use this exact workflow every time:

1. Identify the DP pattern
2. Define the state
3. Identify the choices / last decision
4. Write the recurrence
5. Define base cases
6. Decide computation order
7. Implement (memoization or tabulation)
8. Determine final answer

Step 1 — Identify the Pattern

Ask: What is the shape of my subproblem?

- One sequence $\rightarrow$ likely $dp[i] \rightarrow$ Prefix DP
- Two sequences $\rightarrow dp[i][j] \rightarrow$ Two-Sequence DP
- One range $\rightarrow dp[l][r] \rightarrow$ Interval DP
- Grid $\rightarrow dp[r][c] \rightarrow$ Grid DP
- Items + target $\rightarrow dp[i][target] \rightarrow$ Knapsack DP
- Tree $\rightarrow dp[node] \rightarrow$ Tree DP
- Subset of elements $\rightarrow dp[mask] \rightarrow$ Bitmask DP

Step 2 — Define the State

Ask: “What does $dp[\dots]$ mean?” Write it in plain English before writing any formula.

Common mistakes:

- Confusing the state with the final answer
- Defining too narrow a state (missing information)
- Assuming the shape of state before thinking

Step 3 — Identify the Choices / Last Decision

Ask: “What was the last thing I did before arriving at this state?”

This is the key to deriving the recurrence. The “last decision” approach works for every DP pattern.

- Prefix DP: “What was the last action at position i ?”
- Two-Sequence DP: “What happens to the last characters of both sequences?”
- Interval DP: “What happens to the ends of the interval?” / “Where do I split?”
- Knapsack: “Do I take this item or not?”

Step 4 — Write the Recurrence

After identifying all possible choices, combine them:

Goal	Combine with
Maximize	$\max(\dots)$
Minimize	$\min(\dots)$
Count	$\text{sum}(\dots)$
Possibility	OR / AND

Step 5 — Define Base Cases

Ask: “What are the smallest possible states?” These are typically:

- Empty sequence / empty string $\rightarrow 0$ or True
- Single element $\rightarrow$ direct answer
- Empty interval $i > j \rightarrow 0$

Step 6 — Computation Order

- **Memoization (top-down)**: Natural recursion handles this automatically
- **Tabulation (bottom-up)**: Fill smaller states before larger states

For Prefix DP: left to right. For Interval DP: shorter intervals before longer intervals.

Step 7 — Implement

See the Templates section (§17).

Step 8 — Final Answer

Common mistake: $dp[n-1]$ is not always the answer.

- LIS: $\max(dp[i])$ over all i , not $dp[n-1]$
- Interval DP: usually $dp[0][n-1]$
- Two-Sequence DP: usually $dp[n][m]$
- Prefix DP: usually $dp[n-1]$ or $dp[n]$, depending on definition

Always ask: “Which state represents the full problem?”

6. DP Classification by State Shape

Pattern	Typical State	Main Question
Prefix DP	$dp[i]$	How do I reach i ?
Interval DP (Shrinking)	$dp[i][j]$	What happens to the ends?
Interval DP (Partition)	$dp[i][j] + \text{split } k$	Where should I split?
Two-Sequence DP	$dp[i][j]$	What happens to the last elements?
Grid DP	$dp[r][c]$	Where did I come from?
Knapsack DP	$dp[i][\text{target}]$	Take or don't take?
Tree DP	$dp[\text{node}]$	What do I need from my children?
Bitmask DP	$dp[\text{mask}] / dp[\text{mask}][\text{state}]$	Which elements are already used?

Critical Distinction: When $dp[i][j]$ Appears

Two-Sequence DP:

$i \rightarrow$ position in sequence A

$j \rightarrow$ position in sequence B

Interval DP:

$i \rightarrow$ LEFT boundary of a range

$j \rightarrow$ RIGHT boundary of a range

Both use $dp[i][j]$, but the *meaning* of i and j is completely different. The meaning of the indices determines the pattern, not just their count.

7. Prefix DP

Core idea: Ask what the answer is for the first i elements, processing left to right.

$dp[i]$ = answer for the prefix ending at i

Recognition:

- Processing a sequence from left to right
- The answer at i depends on answers at earlier positions
- You can ask: "How could I have arrived at position i ?"

Possible previous states:

- $i-1$
- $i-2$
- Any $j < i$ satisfying some condition

PREFIX DP

Input: A B C D E
 $\uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow$
 $dp[0] \dots \dots \dots dp[4]$

$dp[4]$
 $\uparrow$
 "How did I get here?"
 $\uparrow$

 combine choices
 max / min / sum

7.1 Climbing Stairs

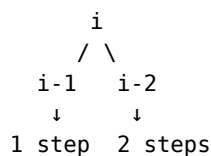
Problem: Climb n stairs. Each step: 1 or 2 stairs. Count the distinct ways to reach the top.

Derivation:

1. **State:** $dp[i]$ = number of ways to reach stair i
2. **Last action:** the final jump is either 1 step or 2 steps
3. **Previous states:** $i-1$ (1-step jump) or $i-2$ (2-step jump)
4. **Recurrence:** counting $\rightarrow$ sum

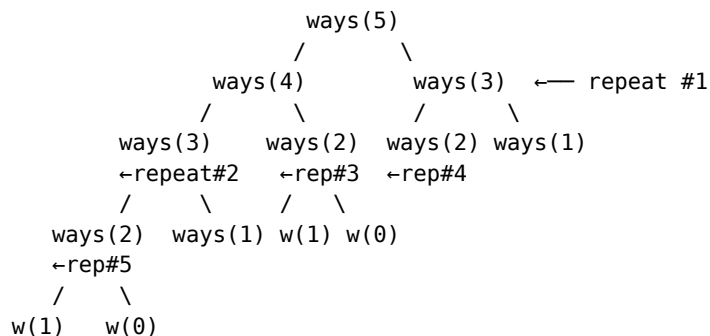
$$dp[i] = dp[i-1] + dp[i-2]$$

To reach stair i :



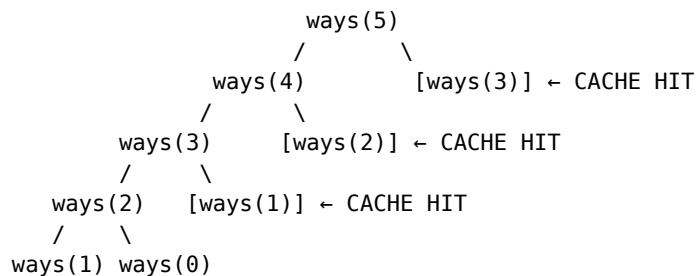
5. **Base cases:** $dp[0] = 1, dp[1] = 1$

Recursion Tree ($n = 5$) — why DP is needed

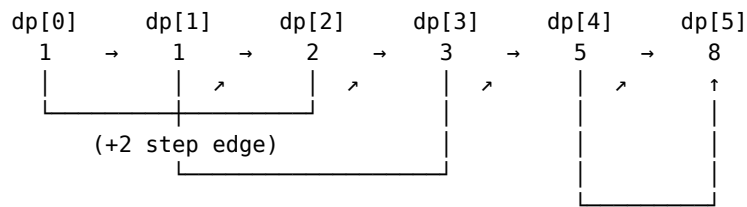


ways(3) computed 2× ways(2) computed 3× ways(1) computed 5×
 Total nodes $\approx 2^n \rightarrow O(2^n)$ without memoization

After memoization — the same tree collapses:



Only 6 distinct states ever computed $\rightarrow O(n)$

State Transition Diagram

Every node receives exactly two incoming edges:

$i-1 \xrightarrow{(1 \text{ step})} i$

$i-2 \xrightarrow{(2 \text{ steps})} i$

and the values are ADDED (we are counting).

Time: $O(n)$ — **Space:** $O(n)$, or $O(1)$ keeping only two previous values.

7.2 House Robber

Problem: Array of house values. Cannot rob adjacent houses. Maximize money robbed.

Derivation:

1. **State:** $dp[i]$ = maximum money robbed from houses $0 \dots i$

2. **Last decision** at house i :

At house i :

Don't rob $i \rightarrow dp[i-1]$

(previous best, no constraint added)

Rob $i \rightarrow dp[i-2] + \text{nums}[i]$

(skip $i-1$, add current house)

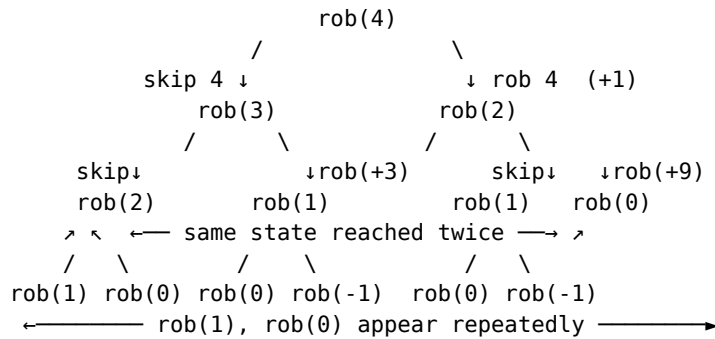
3. **Recurrence:**

$$dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$$

4. **Base cases:** $dp[0] = \text{nums}[0]$, $dp[1] = \max(\text{nums}[0], \text{nums}[1])$

Recursion Tree — $\text{nums} = [2, 7, 9, 3, 1]$

Each node branches into the two decisions. Notice the repeats.

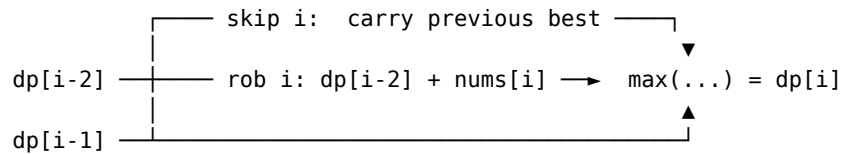


Distinct states: `rob(0)..rob(4)` → only 5

Naive recursion: $O(2^n)$ nodes

With memo: $O(n)$

State Transition Diagram



Two incoming edges, combined with max (we are maximizing).

DP Table Walkthrough — `nums = [2, 7, 9, 3, 1]`

i	0	1	2	3	4
nums[i]	2	7	9	3	1
dp[i]	2	7	11	11	12

$dp[0] = 2$ (base)
 $dp[1] = \max(2, 7) = 7$ (base)
 $dp[2] = \max(dp[1], dp[0] + 9) = \max(7, 11) = 11$ ← rob house 2
 $dp[3] = \max(dp[2], dp[1] + 3) = \max(11, 10) = 11$ ← skip house 3
 $dp[4] = \max(dp[3], dp[2] + 1) = \max(11, 12) = 12$ ← rob house 4

Answer = $dp[4] = 12$ (robbed houses 0, 2, 4 → $2 + 9 + 1$)

Time: $O(n)$ — **Space:** $O(n)$, reducible to $O(1)$.

7.3 Decode Ways

Problem: String of digits. 1→A, 2→B, ..., 26→Z. Count ways to decode s.

Derivation:

1. **State:** $dp[i]$ = number of ways to decode the first i characters
2. **Last action:** decode the last 1 digit, or the last 2 digits

Decoding the first i characters:

... | one digit → contributes $dp[i-1]$
 ... | two digits → contributes $dp[i-2]$

3. Validity conditions:

- One-digit valid if: $s[i-1] \neq '0'$
- Two-digit valid if: $10 \leq \text{int}(s[i-2:i]) \leq 26$

4. Recurrence:

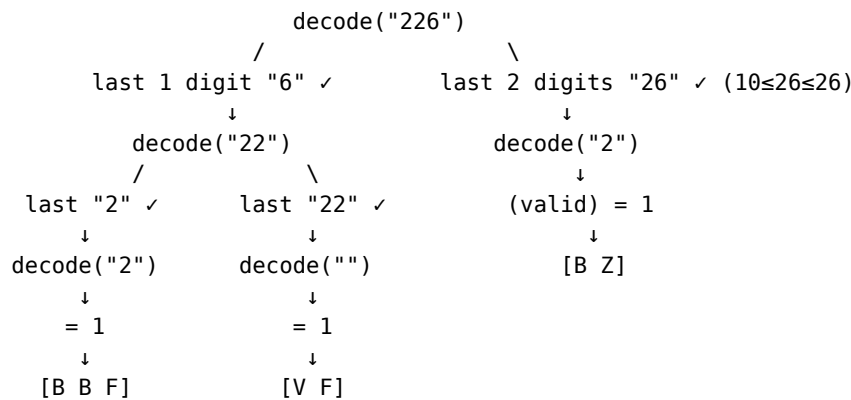
$$dp[i] = \begin{cases} dp[i-1] & \text{if } s[i-1] \neq '0' \\ +dp[i-2] & \text{if } 10 \leq \text{int}(s[i-2:i]) \leq 26 \end{cases}$$

(Both contributions are added independently when their conditions hold.)

5. **Base cases:** $dp[0] = 1, dp[1] = 1$ if $s[0] \neq '0'$ else 0

Recursion Tree — $s = "226"$

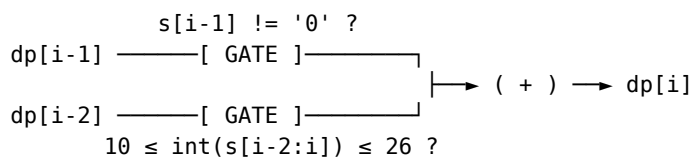
Read the tree top-down as “*peel off the last piece*”:



3 leaves reaching a valid base → answer = 3

Validity Gate Diagram

Each transition is guarded by a condition — this is the new idea here:



If a gate is closed, that edge contributes 0 – not an error, just no path.

DP Table Walkthrough — $s = "226"$

i	0	1	2	3
prefix	"	"2"	"22"	"226"

dp[i]	1	1	2	3
-------	---	---	---	---

dp[0] = 1 (empty string)

dp[1]: $s[0]='2' \neq '0' \rightarrow dp[0] = 1$

dp[2]: $s[1]='2' \neq '0' \rightarrow += dp[1] = 1$

$s[0:2]="22", 10 \leq 22 \leq 26 \rightarrow += dp[0] = 1 \rightarrow \text{total } 2$

dp[3]: $s[2]='6' \neq '0' \rightarrow += dp[2] = 2$

$s[1:3]="26", 10 \leq 26 \leq 26 \rightarrow += dp[1] = 1 \rightarrow \text{total } 3$

Contrast with $s = "06"$:

dp[1]: $s[0]='0' \rightarrow \text{gate closed} \rightarrow dp[1] = 0$

dp[2]: $s[1]='6' \neq '0' \rightarrow += dp[1] = 0$

$s[0:2]="06" = 6, \text{ not in } [10, 26] \rightarrow \text{gate closed}$

$\rightarrow dp[2] = 0$

Key insight: DP transitions can depend on validity conditions, not just fixed index offsets. This problem teaches that each transition is conditional.

Time: $O(n)$ — **Space:** $O(n)$, reducible to $O(1)$.

7.4 Coin Change

Problem: Array of coin denominations. Make amount using the minimum number of coins. Coins can be reused.

Derivation:

1. **State:** $dp[i]$ = minimum coins needed to make amount i

2. **Last action:** add one coin c to a previous amount

dp[amount]

↑

for each coin c :

$dp[\text{amount} - c] + 1$

3. **Recurrence:**

$$dp[x] = \min_{c \in \text{coins}, c \leq x} (dp[x - c] + 1)$$

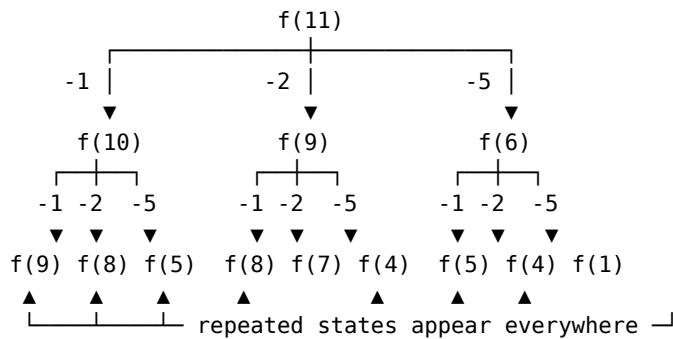
The +1 accounts for the coin c we just added. This is not $i-1$ or $i-2$ fixed — we look at any $x - c$ for any valid coin.

4. **Base case:** $dp[0] = 0$

5. **Initialize:** $dp[i] = \text{infinity}$ for all $i > 0$

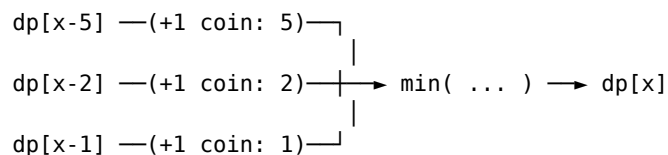
Recursion Tree — coins = [1, 2, 5], amount = 11

Here the branching factor is the number of coins, not a fixed 2:



$f(9)$ appears twice at depth 2. $f(8)$ twice. $f(5)$, $f(4)$ twice. Deeper down the overlap explodes.

Distinct states: only 0..11 $\rightarrow$ 12 states.

State Transition Diagram

One incoming edge per coin. Combine with min (we are minimizing). The "+1" on every edge is the coin we just placed.

DP Table Walkthrough — coins = [1, 2, 5], amount = 11

x	0	1	2	3	4	5	6	7	8	9	10	11
dp	0	1	1	2	2	1	2	2	3	3	2	3

```

dp[1] = min(dp[0]+1) = 1 [1]
dp[2] = min(dp[1]+1, dp[0]+1) = 1 [2]
dp[3] = min(dp[2]+1, dp[1]+1) = 2 [2,1]
dp[5] = min(dp[4]+1, dp[3]+1, dp[0]+1) = 1 [5]
dp[11] = min(dp[10]+1, dp[9]+1, dp[6]+1)
        = min( 2+1, 3+1, 2+1 ) = 3 [5,5,1]

```

Time: $O(\text{amount} \times |\text{coins}|)$ — **Space:** $O(\text{amount})$

7.5 Longest Increasing Subsequence (LIS)

Problem: Find the length of the longest strictly increasing subsequence.

Derivation:

1. **State:** $dp[i]$ = LIS ending at index i
2. **Last action:** extend from any valid earlier element

For $\text{nums}[i]$, which previous elements can we extend?

Answer: any $j < i$ where $\text{nums}[j] < \text{nums}[i]$

3. Recurrence:

$$dp[i] = \max_{j < i, \text{nums}[j] < \text{nums}[i]} (dp[j] + 1)$$

If no valid j exists: $dp[i] = 1$

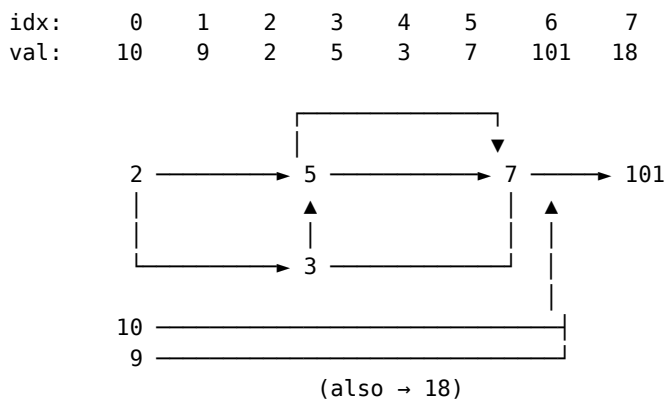
4. **Important:** This is not fixed $i-1$ or $i-2$. Any earlier position can contribute.

5. **Base case:** $dp[i] = 1$ for all i (each element alone is an LIS of length 1)

6. **Final answer:** $\max(dp[i])$ over all i

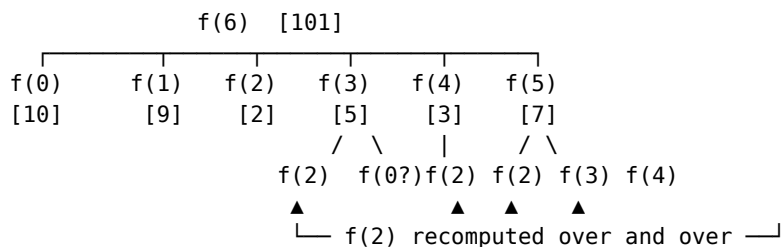
State Transition Diagram — $\text{nums} = [10, 9, 2, 5, 3, 7, 101, 18]$

Draw an edge $j \rightarrow i$ whenever $j < i$ and $\text{nums}[j] < \text{nums}[i]$. This is a DAG, and $dp[i]$ is the longest path ending at i :



Longest chain: $2 \rightarrow 3 \rightarrow 7 \rightarrow 101$ (length 4)

Recursion Tree — $f(i) = \text{LIS ending at } i$



Each $f(i)$ may call every $f(j)$ for $j < i \rightarrow O(2^n)$ naive.

With memo: n states $\times O(n)$ work each $\rightarrow O(n^2)$

DP Table Walkthrough

idx	0	1	2	3	4	5	6	7
val	10	9	2	5	3	7	101	18
dp	1	1	1	2	2	3	4	4

▲
 $\max(dp) = 4 \leftarrow \text{THE ANSWER}$

$dp[3]$ (val 5): $j=2$ ($2 < 5$) $\rightarrow dp[2]+1 = 2$
 $dp[4]$ (val 3): $j=2$ ($2 < 3$) $\rightarrow dp[2]+1 = 2$
 $dp[5]$ (val 7): $j=2 \rightarrow 2, j=3 \rightarrow 3, j=4 \rightarrow 3 \rightarrow 3$
 $dp[6]$ (101): $j=5 \rightarrow dp[5]+1 = 4 \rightarrow 4$
 $dp[7]$ (18): $j=5 \rightarrow dp[5]+1 = 4 \rightarrow 4$

Note $dp[7] = 4$ but $dp[6] = 4$ too – the answer is NOT the last cell.

Common mistake: $dp[n-1]$ is NOT necessarily the answer. The LIS may end at any index. Always take $\max(dp)$.

Time: $O(n^2)$ — **Space:** $O(n)$

7.6 Word Break

Problem: Given string s and dictionary $wordDict$, determine if s can be segmented into dictionary words.

Derivation:

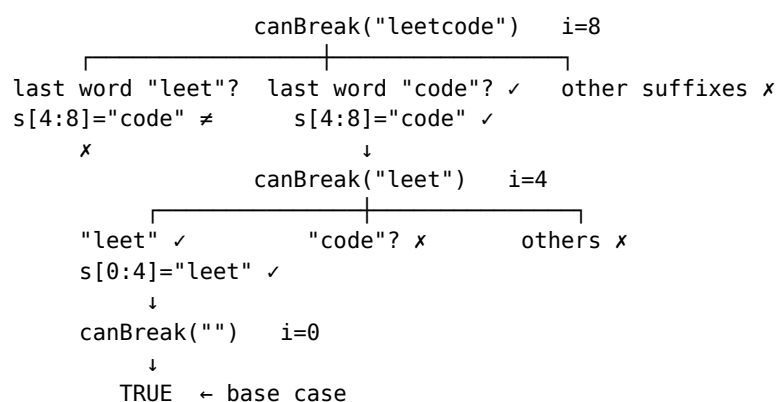
1. **State:** $dp[i]$ = can the first i characters be segmented?
2. **Last action:** the last dictionary word covers some suffix

[first $i - word $ characters]		[last word]
$dp[i - word]$		word
✓		✓

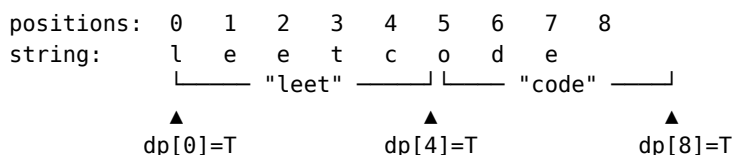
3. **Recurrence:**

$$dp[i] = \exists \text{ word} \in \text{wordDict} : dp[i - |\text{word}|] \wedge s[i - |\text{word}| : i] = \text{word}$$

4. **Base case:** $dp[0] = \text{True}$ (empty prefix is trivially segmentable)

Recursion Tree — $s = \text{"leetcode"}, \text{wordDict} = [\text{"leet"}, \text{"code"}]$ 

Path found: "leet" | "code" → $dp[8] = \text{True}$

State Transition Diagram — cut-point view

To compute $dp[i]$, scan every earlier cut point p :

$dp[p] \text{ AND } s[p:i] \in \text{dict} ? \rightarrow \text{contributes True to } dp[i]$

Combine all cut points with OR (we only need ONE valid segmentation).

DP Table Walkthrough — $s = \text{"leetcode"}$

i	0	1	2	3	4	5	6	7	8
prefix	""	"l"	"le"	"lee"	"leet"	...	...	...	"leetcode"
dp	T	F	F	F	T	F	F	F	T
	▲ base				▲ "leet" ✓				▲ "code" ✓

$dp[4] = dp[0] \text{ AND } (\text{"leet" in dict}) = T \text{ AND } T = \text{True}$

$dp[8] = dp[4] \text{ AND } (\text{"code" in dict}) = T \text{ AND } T = \text{True}$

All other cells stay False — no valid word ends there.

Key insight: Instead of looking at fixed $i-1$ or $i-2$, we try every possible dictionary word as the last piece. This is still Prefix DP — the state is $dp[i]$ and we process left to right.

Time: $O(n^2 \cdot L)$ where L is average word length — **Space:** $O(n)$

8. Two-Sequence DP

Core idea: When two sequences must be compared, aligned, or transformed, the natural state involves one index per sequence.

Sequence A → index i

Sequence B → index j

$dp[i][j]$ = answer using the first i elements of A
and the first j elements of B

Recognition trigger: Two strings / arrays + comparison / alignment / matching / transformation → Two-Sequence DP.

Central question: “What happens to the **last elements** of the two sequences?”

Two sequences:

A: a b c d e
B: a c e

$dp[i][j]$

↓

Look at $A[i-1]$, $B[j-1]$

↓

Match	Don't match
diagonal $dp[i-1][j-1]$	discard one $\max(\text{top}, \text{left})$

8.1 Longest Common Subsequence (LCS)

Problem: Find length of the longest common subsequence of two strings.

Derivation:

1. **State:** $dp[i][j]$ = LCS of $\text{text1}[0 \dots i-1]$ and $\text{text2}[0 \dots j-1]$
2. **Last characters:** consider $\text{text1}[i-1]$ and $\text{text2}[j-1]$

If they match ($\text{text1}[i-1] == \text{text2}[j-1]$):

text1 : ... A
 text2 : ... A
↓ remove both, solve smaller problem
 $dp[i-1][j-1]$
↓ include this matching pair
+ 1

$$dp[i][j] = dp[i-1][j-1] + 1$$

If they don't match:

Option A: discard $\text{text1}[i-1]$ → $dp[i-1][j]$

Option B: discard $\text{text2}[j-1]$ → $dp[i][j-1]$

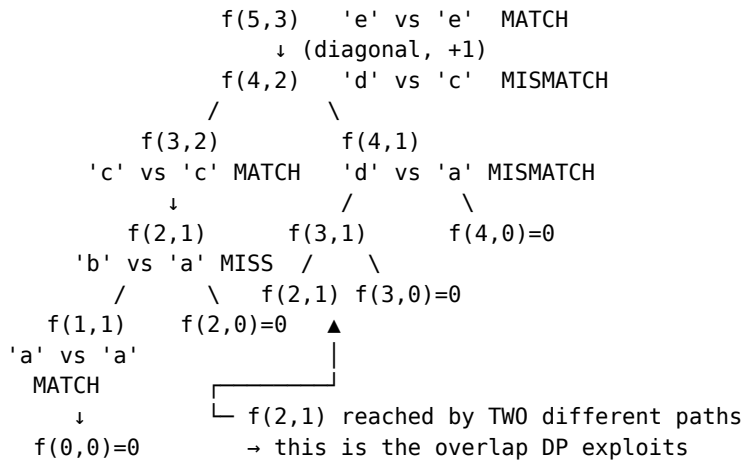
Take the better:

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

The two mismatch choices are necessary because either character might be useful in future matches — we keep the option that preserves the most.

3. **Base cases:** $dp[0][j] = 0, dp[i][0] = 0$ (empty sequence has no common subsequence)

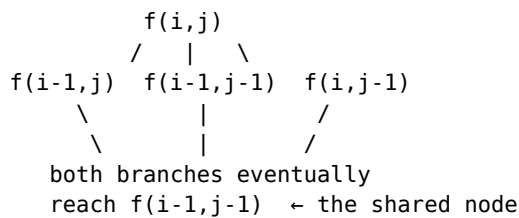
Recursion Tree — `text1 = "abcde", text2 = "ace"`



Naive: $O(2^{(n+m)})$ calls

Memo: only $(n+1)(m+1) = 6 \times 4 = 24$ distinct states

Where the overlap comes from visually:



State Transition Diagram

MATCH: `text1[i-1] == text2[j-1]`

$dp[i-1][j-1] \xrightarrow{(+1)} dp[i][j]$
 ↘ (diagonal only – one edge)

MISMATCH: `text1[i-1] != text2[j-1]`

$dp[i-1][j]$ — (drop char from text1) —→ $dp[i][j]$
 $dp[i][j-1]$ — (drop char from text2) —→ $dp[i][j]$
 ↑ (up) ← (left)

max

DP Table Walkthrough — `"abcde" vs "ace"`

Diagonal arrows mean “match, +1”. Up/left arrows mean “take the better”.

	" "	a	c	e	
" "	0	0	0	0	
a	0	↖1	←1	←1	a==a → 0+1
b	0	↑1	←1	←1	no match anywhere
c	0	↑1	↖2	←2	c==c → 1+1
d	0	↑1	↑2	←2	no match
e	0	↑1	↑2	↖3	e==e → 2+1

▲
answer = dp[5][3] = 3

Trace back along ↖ arrows: e ← c ← a → LCS = "ace"

Time: $O(mn)$ — **Space:** $O(mn)$, reducible to $O(\min(m, n))$ with two rows.

8.2 Edit Distance

Problem: Minimum operations (Insert, Delete, Replace) to convert word1 into word2.

All operations are performed **on word1** to make it become word2.

Derivation:

1. **State:** $dp[i][j]$ = minimum operations to convert $word1[:i] \rightarrow word2[:j]$
2. **If characters match ($word1[i-1] == word2[j-1]$):** no operation needed:

$$dp[i][j] = dp[i-1][j-1]$$

3. **If characters don't match:** three operations:

Operation	What it does	Previous state
Delete from word1	Remove $word1[i-1]$	$dp[i-1][j]$
Insert into word1	Add needed char from word2	$dp[i][j-1]$
Replace in word1	Swap $word1[i-1]$ for $word2[j-1]$	$dp[i-1][j-1]$

Why $dp[i-1][j]$ is delete (not insert): $dp[i-1][j]$ means we've "removed" one character from word1 side — we've consumed one fewer character of word1 — that's a deletion.

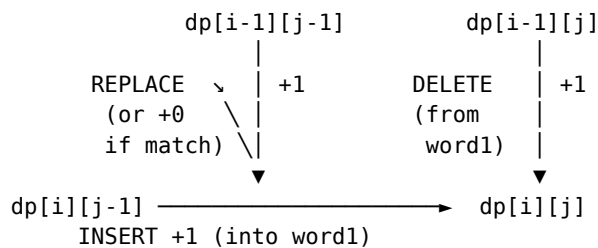
$$dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$$

4. **Base cases:**

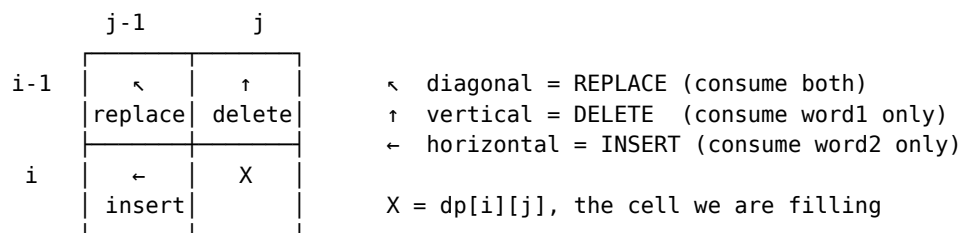
- $dp[i][0] = i$ (delete all i chars from word1 to reach empty word2)
- $dp[0][j] = j$ (insert j chars into word1 to reach word2[:j])

State Transition Diagram — the three operations

Each operation is an arrow *into* $dp[i][j]$, and each costs 1:



Reading the grid geometrically:

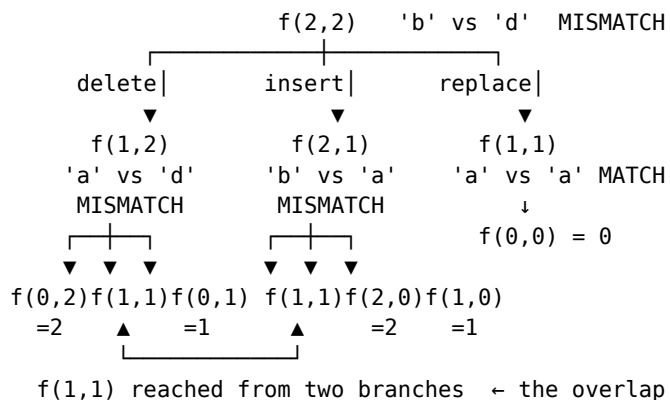


How each operation moves the state:

word1 = "ab", word2 = "ad", at $dp[2][2]$, 'b' ≠ 'd'

DELETE b: "ab" → "a" still need "a" → "ad" → $dp[1][2] + 1$
 INSERT d: "ab" → "abd" the d is supplied → $dp[2][1] + 1$
 REPLACE b→d: "ab" → "ad" done with both chars → $dp[1][1] + 1$

Recursion Tree — word1 = "ab", word2 = "ad"



Best path: replace b→d → $1 + f(1,1) = 1 + 0 = 1$

Because with repeated characters (e.g., $s = \text{"aaa"}, t = \text{"aa"}$), there are *multiple* ways to choose which a s to use. Skipping is always a valid option. Forcing “use every match” would miss distinct ways.

3. If characters don't match ($s[i-1] \neq t[j-1]$):

We cannot use $s[i-1]$ to match $t[j-1]$. Only option: skip $s[i-1]$.

$$dp[i][j] = dp[i-1][j]$$

Summary:

Match $\rightarrow$ USE or SKIP $\rightarrow dp[i-1][j-1] + dp[i-1][j]$

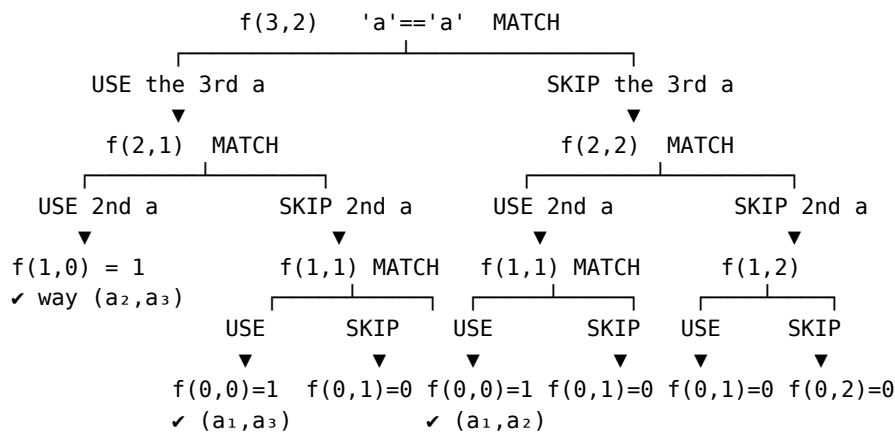
Mismatch $\rightarrow$ SKIP only $\rightarrow dp[i-1][j]$

Why sum instead of max/min? Because we are *counting* all distinct ways, not optimizing one. Every valid choice contributes its own count independently.

4. **Base cases:** $dp[i][0] = 1$ (empty t matched by any s), $dp[0][j] = 0$ for $j > 0$ (no s can match a non-empty t)

Recursion Tree — $s = \text{"aaa"}, t = \text{"aa"}$

This tree makes the “why sum, not max” point concrete. Every leaf reaching a base case is a *distinct way*, and we count them all:



Total valid leaves = 3 $\rightarrow (a_1, a_2), (a_1, a_3), (a_2, a_3)$

This is exactly why the matching case is a SUM:

If we kept ONLY $dp[i-1][j-1]$ (the "USE" branch):
 we would force every matching character to be used
 $\rightarrow$ we would find only 1 way, not 3.

The "SKIP" branch $dp[i-1][j]$ is what discovers (a_1, a_2) – the way that never uses the last a .

Prefix DP vs. Interval DP

Prefix DP:	Interval DP:
$dp[i]$	$dp[i][j]$
$i \rightarrow$ prefix end	$i \rightarrow$ LEFT boundary
	$j \rightarrow$ RIGHT boundary

10. Shrinking Interval DP

Central question: What happens to the two ends $s[i]$ and $s[j]$?

```
[i ..... j]
  ↓         ↓
left      right
examine the ends
  ↓
ends match → shrink inward:  $dp[i+1][j-1]$ 
ends differ → skip left OR skip right
```

Computation order: solve smaller intervals (shorter lengths) before larger ones.

Longest Palindromic Subsequence (LPS)

Problem: Find the length of the longest palindromic subsequence in s .

Derivation:

1. **State:** $dp[i][j]$ = length of the longest palindromic subsequence in $s[i \dots j]$
2. **Look at the ends:**

Case 1: $s[i] == s[j]$

```
 $s[i] \dots s[j]$  ← both ends match
  ↓ include both in palindrome
 $2 + dp[i+1][j-1]$ 
```

$$dp[i][j] = 2 + dp[i+1][j-1]$$

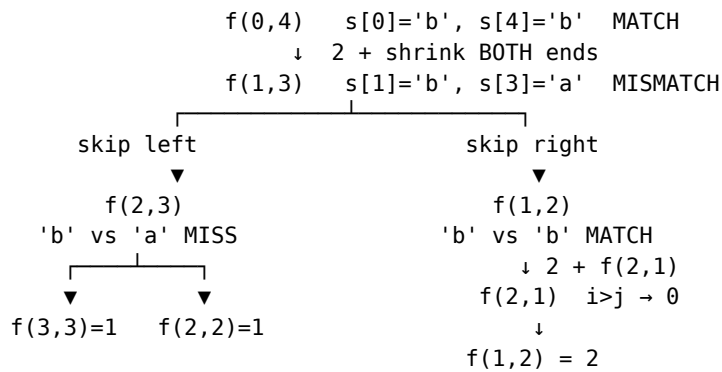
Case 2: $s[i] != s[j]$

Cannot use both ends as palindrome pair.
 Try without left end: $dp[i+1][j]$
 Try without right end: $dp[i][j-1]$
 Take the better:

$$dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$$

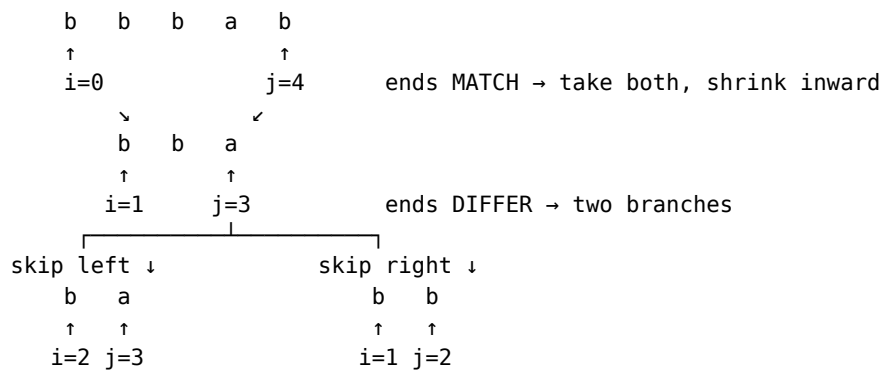
3. Base cases:

- $dp[i][i] = 1$ (single character is a palindrome of length 1)
- $dp[i][j] = 0$ when $i > j$ (empty interval)

Recursion Tree — $s = \text{"bbbab"}$ (indices 0..4)

$$f(0,4) = 2 + \max(f(2,3), f(1,2)) = 2 + \max(1, 2) = 4$$

Answer = 4 ("bbbbb")

The "Shrinking" Motion, Visualized

Every recursive call makes the interval strictly SMALLER.

No split point k is ever chosen – that is what makes this "shrinking".

DP Table Walkthrough — $s = \text{"bbbab"}$

Interval DP fills the table by **increasing length**, so only the upper triangle is used:

j →	0(b)	1(b)	2(b)	3(a)	4(b)	
i=0(b)	1	2	3	3	4	← answer at dp[0][4]
i=1(b)	—	1	2	2	3	
i=2(b)	—	—	1	1	3	
i=3(a)	—	—	—	1	1	
i=4(b)	—	—	—	—	1	

Fill order (by length):

len 1: (0,0)(1,1)(2,2)(3,3)(4,4) ← the diagonal, all = 1
 len 2: (0,1)(1,2)(2,3)(3,4)
 len 3: (0,2)(1,3)(2,4)
 len 4: (0,3)(1,4)
 len 5: (0,4) ← the answer

dp[2][4]: s[2]='b', s[4]='b' match → 2 + dp[3][3] = 2 + 1 = 3

dp[0][4]: s[0]='b', s[4]='b' match → 2 + dp[1][3] = 2 + 2 = 4

Why fill by length? dp[i][j] needs dp[i+1][j-1], dp[i+1][j], dp[i][j-1] — all shorter intervals. Iterating by length guarantees they exist.

Time: $O(n^2)$ — **Space:** $O(n^2)$

11. Partition DP

Recognition rule:

If the problem asks **where to split, partition, cut, divide, or parenthesize** an interval → think Partition DP.

Central question: “Where should I split [i, j]?”

Generic transition:

[i ————— j]

Try every k:

[i — k] | [k+1 — j]

$$dp[i][j] = \min_{k=i}^{j-1} (dp[i][k] + dp[k+1][j] + \text{cost}(i, k, j))$$

Shrinking vs. Partition — clear distinction:

Shrinking Interval:

[l r]

↓

[l+1 .. r-1] ← look at ends, shrink inward

Question: "What happens to the ends?"

Example: Longest Palindromic Subsequence

Partition:

[l r]

k

[l k] [k+1 .. r] ← try every split

Question: "Where should I split?"

Example: Matrix Chain Multiplication

12. Matrix Chain Multiplication (MCM)

The Problem (Understanding First)

You have matrices $A_1 \times A_2 \times A_3 \times \dots \times A_n$. Matrix multiplication is **associative** — the result is the same regardless of parenthesization — but the **cost** (number of scalar multiplications) depends on the order.

Example:

$A = 10 \times 20$

$B = 20 \times 30$

$C = 30 \times 40$

Option 1: $(A \times B) \times C$

$A \times B$ costs: $10 \times 20 \times 30 = 6,000$ → result: 10×30

$(A \times B) \times C$: $10 \times 30 \times 40 = 12,000$

Total = 18,000

Option 2: $A \times (B \times C)$

$B \times C$ costs: $20 \times 30 \times 40 = 24,000$ → result: 20×40

$A \times (B \times C)$: $10 \times 20 \times 40 = 8,000$

Total = 32,000

Same final matrix. Very different cost. We want to find the **minimum-cost parenthesization**.

Why does $p \times q \times r$ appear?

For $A (p \times q) \times B (q \times r)$:

- Result is $p \times r$
- Each of the $p \times r$ output elements requires q multiplications
- Total: $p \times r \times q = p \times q \times r$

DP Derivation

Classification: Interval DP → Partition (we're asking "where to split")

State: $dp[i][j]$ = minimum multiplication cost for multiplying matrices A_i through A_j

Dimension array: Matrices are represented as:

```
dims = [10, 20, 30, 40]
```

```
A1 = dims[0] × dims[1] = 10 × 20
```

```
A2 = dims[1] × dims[2] = 20 × 30
```

```
A3 = dims[2] × dims[3] = 30 × 40
```

In general: $A_i = \text{dims}[i-1] \times \text{dims}[i]$

Why $\text{dims}[i-1]$? Matrix numbering starts at 1, array indexing at 0. So A_1 gets its rows from $\text{dims}[0]$, i.e., $\text{dims}[1-1]$.

Split at k :

$A_i \dots A_k \mid A_{k+1} \dots A_j$

Left result: $\text{dims}[i-1] \times \text{dims}[k]$

Right result: $\text{dims}[k] \times \text{dims}[j]$

Merge cost: $\text{dims}[i-1] \times \text{dims}[k] \times \text{dims}[j]$

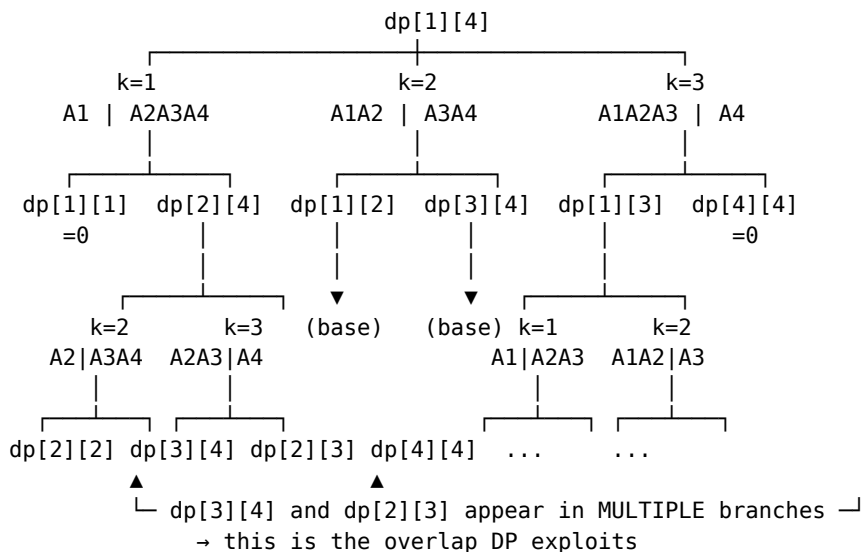
Recurrence:

$$dp[i][j] = \min_{k=i}^{j-1} (dp[i][k] + dp[k+1][j] + \text{dims}[i-1] \times \text{dims}[k] \times \text{dims}[j])$$

Base case: $dp[i][i] = 0$ (single matrix, no multiplication needed)

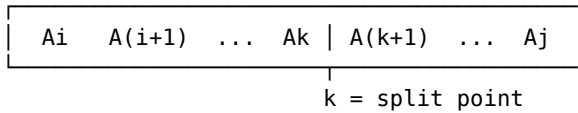
Recursion Tree — $dp[1][4]$ (four matrices $A_1..A_4$)

Unlike shrinking interval DP (one or two branches), Partition DP branches **once per split point**:



The Interval / Split-Point Picture

$dp[i][j]$ = the chain $A_i \dots A_j$



LEFT block $\rightarrow$ solved recursively $\rightarrow dp[i][k]$
 resulting matrix: $\text{dims}[i-1] \times \text{dims}[k]$

RIGHT block $\rightarrow$ solved recursively $\rightarrow dp[k+1][j]$
 resulting matrix: $\text{dims}[k] \times \text{dims}[j]$

MERGE the two results:
 $(\text{dims}[i-1] \times \text{dims}[k]) \times (\text{dims}[k] \times \text{dims}[j])$
 $\downarrow$
 $\text{cost} = \underset{p}{\text{dims}[i-1]} \times \underset{q}{\text{dims}[k]} \times \underset{r}{\text{dims}[j]}$

Worked Example — $\text{dims} = [10, 20, 30, 40]$, three matrices

$A_1 = 10 \times 20$ $A_2 = 20 \times 30$ $A_3 = 30 \times 40$

$dp[1][3]$ – try every k :

$k = 1$: $A_1 | A_2 A_3$
 $dp[1][1] + dp[2][3] + \text{dims}[0] \cdot \text{dims}[1] \cdot \text{dims}[3]$
 $= 0 + 24,000 + 10 \cdot 20 \cdot 40$
 $= 0 + 24,000 + 8,000 = 32,000$

$k = 2$: $A_1 A_2 | A_3$
 $dp[1][2] + dp[3][3] + \text{dims}[0] \cdot \text{dims}[2] \cdot \text{dims}[3]$
 $= 6,000 + 0 + 10 \cdot 30 \cdot 40$
 $= 6,000 + 0 + 12,000 = 18,000 \leftarrow \text{MINIMUM}$

$dp[1][3] = \min(32,000, 18,000) = 18,000$

Notice this reproduces the hand-computed answer from the very first example: $(A \times B) \times C = 18,000$ beats $A \times (B \times C) = 32,000$.

DP Table + Computation Order

$j \rightarrow$	1	2	3	
$i=1$	0	6,000	18,000	$\leftarrow$ answer at $dp[1][n]$
$i=2$	–	0	24,000	
$i=3$	–	–	0	

Fill order (length = chain size):

len 1: $dp[1][1]$ $dp[2][2]$ $dp[3][3]$	all = 0 (diagonal)
len 2: $dp[1][2]$ $dp[2][3]$	one k each
len 3: $dp[1][3]$	two k values $\leftarrow$ answer

$dp[1][2] = 10 \cdot 20 \cdot 30 = 6,000$
 $dp[2][3] = 20 \cdot 30 \cdot 40 = 24,000$

Computation order: solve smaller intervals (shorter $j - i$) before larger ones. Use the Partition interval tabulation template.

Time: $O(n^3) - O(n^2)$ states $\times O(n)$ split points each. **Space:** $O(n^2)$

13. Grid DP

Recognition:

matrix / grid / board
 +
 movement rules (right, down, diagonal, ...)
 +
 answer associated with reaching / leaving cells
 → Grid DP

Core state:

$dp[r][c]$ = answer for the cell at row r , column c

Central question: “Where could I have come from?”

If movement is only right ($\rightarrow$) and down ($\downarrow$):

from top
 $\downarrow$
 $[r-1, c]$

$(r, c-1) \rightarrow [r, c]$

Previous cells are $(r-1, c)$ and $(r, c-1)$.

13.1 Minimum Path Sum

State: $dp[r][c]$ = minimum path sum from $(0, 0)$ to (r, c)

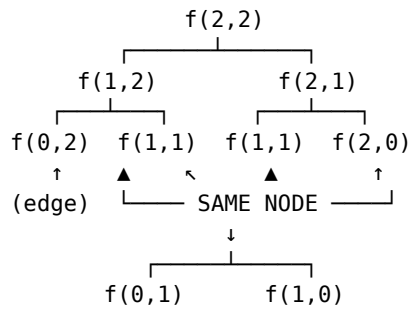
Recurrence:

$$dp[r][c] = \text{grid}[r][c] + \min(dp[r-1][c], dp[r][c-1])$$

Why min? We want the cheapest route, so we pick the cheaper predecessor.

Base cases:

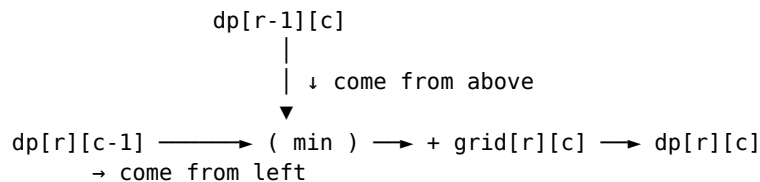
- $dp[0][0] = \text{grid}[0][0]$
- First row: $dp[0][c] = dp[0][c-1] + \text{grid}[0][c]$ (can only come from left)
- First column: $dp[r][0] = dp[r-1][0] + \text{grid}[r][0]$ (can only come from above)

Recursion Tree — 3×3 grid, $f(r, c) = \text{min cost to reach } (r, c)$ 

$f(1,1)$ is reached from BOTH $f(1,2)$ and $f(2,1)$.

In a bigger grid the number of paths to a cell grows like $C(r+c, r)$

→ exponential recursion, but only $r \times c$ distinct states.

State Transition Diagram

Two incoming edges → take the CHEAPER one → then pay the current cell.

DP Table Walkthrough

grid:

1	3	1
1	5	1
4	2	1

→

dp:

1	4	5
2	7	6
6	8	7

first row: only from left

← answer = 7



$dp[0][0] = 1$

first row : $1 \rightarrow 1+3=4 \rightarrow 4+1=5$

first col : $1 \rightarrow 1+1=2 \rightarrow 2+4=6$

$dp[1][1] = 5 + \min(dp[0][1]=4, dp[1][0]=2) = 5 + 2 = 7$

$dp[1][2] = 1 + \min(dp[0][2]=5, dp[1][1]=7) = 1 + 5 = 6$

$dp[2][1] = 2 + \min(dp[1][1]=7, dp[2][0]=6) = 2 + 6 = 8$

$dp[2][2] = 1 + \min(dp[1][2]=6, dp[2][1]=8) = 1 + 6 = 7$

Optimal path: $1 \rightarrow 3 \rightarrow 1 \rightarrow 1 \rightarrow 1$ (right, right, down, down)

Time: $O(rc)$ — **Space:** $O(rc)$, reducible to $O(c)$ with one row.

13.2 Unique Paths

State: $dp[r][c]$ = number of paths from $(0,0)$ to (r,c)

Recurrence:

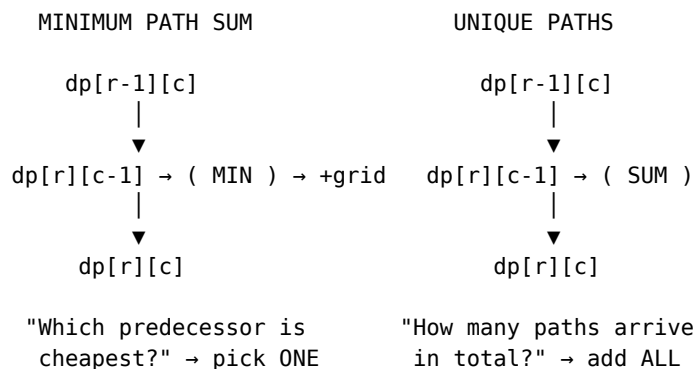
$$dp[r][c] = dp[r-1][c] + dp[r][c-1]$$

Why sum instead of min? We are *counting* all valid paths. Every path from above and every path from the left independently reaches (r, c) .

Problem	Transition	Why
Minimum Path Sum	$grid[r][c] + \min(\dots)$	Pick cheapest predecessor
Unique Paths	$\dots + \dots$	Count all predecessors

Base cases: $dp[0][j] = 1, dp[i][0] = 1$ (only one way to travel along edges)

State Transition Diagram — same grid, different combiner

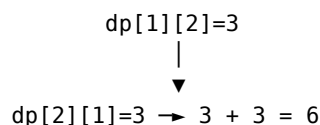


This is the single most important contrast in Grid DP. The *structure* is identical; only the combiner changes because the *question* changed from optimizing to counting.

DP Table Walkthrough — 3×3 grid

	c=0	c=1	c=2	
r=0	1	1	1	first row: 1 way (all right)
r=1	1	2	3	2 = 1+1, 3 = 1+2
r=2	1	3	6	3 = 1+2, 6 = 3+3 ← answer

Arrow view of $dp[2][2]$:



Time: $O(rc)$ — **Space:** $O(rc)$, reducible to $O(c)$.

13.3 Maximal Square

Problem: Find the largest square of all 1s in a binary matrix. Return its area.

State: $dp[r][c]$ = **side length** of the largest square whose bottom-right corner is (r, c)

The local 2x2 relationship:

A B
C X (X is the current cell at (r, c))

For a square to end at X:

- A = $dp[r-1][c-1]$ (diagonal)
- B = $dp[r-1][c]$ (top)
- C = $dp[r][c-1]$ (left)

Why min? The square can only be as large as the *weakest* supporting direction. If top supports a 3x3 but left only supports a 1x1, the square is bottlenecked at 1x1.

If $X = 1$:

$$dp[r][c] = 1 + \min(dp[r-1][c], dp[r][c-1], dp[r-1][c-1])$$

Why +1?

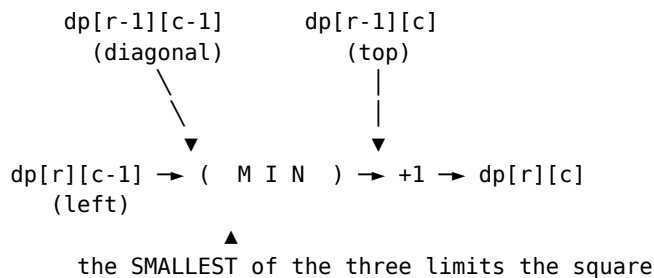
- $\min(\text{top}, \text{left}, \text{diagonal})$ tells us how big a square the neighbors can support
- The current cell X extends that square by one row and one column
- So we add 1 for the new layer

If $X = 0$:

$$dp[r][c] = 0$$

Final answer: $\max(dp[r][c])^2$ over all cells (side length squared = area)

State Transition Diagram — the bottleneck



Why $\min$ is a Bottleneck — visual proof

Suppose the three neighbours report:

diagonal = 3 top = 2 left = 4

Top can only guarantee 2 rows of support:

```

? ? ? ?      The 4th row up is NOT guaranteed to be all 1s,
1 1 1 ?      because top's value is 2, not 3.
1 1 1 ?
1 1 1 X  ← X is our cell

```

So no matter how good diagonal and left are,
the square is capped at $\min(3, 2, 4) = 2$
and X extends it by one layer: $dp = 2 + 1 = 3$

Why +1 — one new layer

neighbours support
a square of side 2

```

1 1
1 1

```

side 2

X completes the
next layer

```

1 1 | 1
1 1 | 1
-----
1 1 | X

```

side 2 + 1 = 3

DP Table Walkthrough

matrix:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

→

dp (side lengths):

1	0	1	0	0
1	0	1	1	1
1	1	1	2	2
1	0	0	1	0

▲
max = 2 → area = 4

dp[2][3]: cell is 1

top = dp[1][3] = 1

left = dp[2][2] = 1

diagonal = dp[1][2] = 1

→ 1 + min(1,1,1) = 2 ✓ the 2×2 square exists

dp[2][4]: cell is 1

top = dp[1][4] = 1

left = dp[2][3] = 2

diagonal = dp[1][3] = 1

→ 1 + min(1,2,1) = 2 ← top and diagonal bottleneck it

Time: $O(rc)$ — **Space:** $O(rc)$, reducible to $O(c)$.

14. Knapsack DP

Core question:

Do I take this item, or don't I?

Recognition words: subset, partition, target, capacity, weight, choose items, take-or-skip.

14.1 Partition Equal Subset Sum (0/1 Knapsack)

Problem: Can we split nums into two subsets with equal sums?

Step 0 — Reduce the problem: Total must be even. If total is even, we need: can we find a subset summing to total / 2?

```
nums = [1, 5, 11, 5]
total = 22
target = 11
```

State: $dp[i][s]$ = can we form sum s using the first i elements?

Last decision for element $x = \text{nums}[i-1]$:

```
Don't take x → can first i-1 elements make s?    → dp[i-1][s]
Take x      → can first i-1 elements make s-x?    → dp[i-1][s-x]
```

Recurrence:

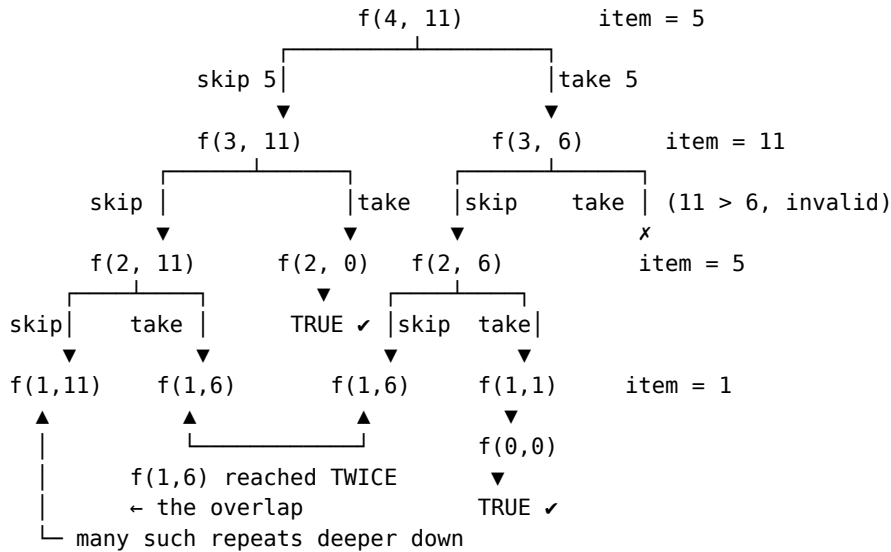
$$dp[i][s] = dp[i-1][s] \text{ OR } dp[i-1][s-x] \quad (\text{when } x \leq s)$$

Both options use $i-1$ because this is **0/1 Knapsack** — each element used at most once.

Base cases: $dp[0][0] = \text{True}$, $dp[0][s] = \text{False}$ for $s > 0$

Recursion Tree — $\text{nums} = [1, 5, 11, 5]$, $\text{target} = 11$

Every level is one item; every node branches into TAKE / DON'T TAKE:



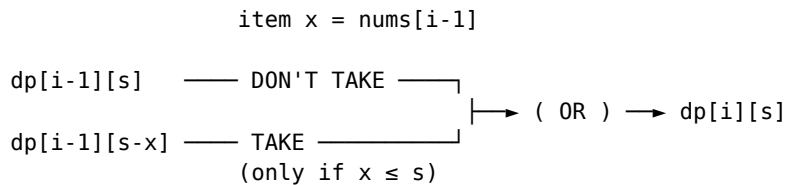
Two TRUE leaves found:

{11} → skip 5, skip 5, take 11

{1, 5, 5} → take 5, skip 11, take 5, take 1

Answer: True

State Transition Diagram



Both edges come from row $i-1$ ← this is what makes it 0/1.

If an edge came from row i , the same item could be taken again

→ that would be UNBOUNDED knapsack.

2D DP Table Walkthrough — $nums = [1, 5, 11, 5]$, $target = 11$

s →	0	1	2	3	4	5	6	7	8	9	10	11
i=0 {}	T	F	F	F	F	F	F	F	F	F	F	F
i=1 +1	T	T	F	F	F	F	F	F	F	F	F	F
i=2 +5	T	T	F	F	F	T	T	F	F	F	F	F
i=3 +11	T	T	F	F	F	T	T	F	F	F	F	T
i=4 +5	T	T	F	F	F	T	T	F	F	F	T	T

▲
dp[4][11] = True ✓

Row $i=2$ (adding 5): $dp[2][5] = dp[1][5] \text{ OR } dp[1][0] = F \text{ OR } T = T$

$dp[2][6] = dp[1][6] \text{ OR } dp[1][1] = F \text{ OR } T = T$

Row $i=3$ (adding 11): $dp[3][11] = dp[2][11] \text{ OR } dp[2][0] = F \text{ OR } T = T$

Row $i=4$ (adding 5): $dp[4][10] = dp[3][10] \text{ OR } dp[3][5] = F \text{ OR } T = T$

Time: $O(n \times \text{target})$ — **Space:** $O(n \times \text{target})$

14.2 1D Optimization

Since $\text{dp}[i]$ only depends on $\text{dp}[i-1]$, we can reduce to a 1D array:

```
dp[s] = True/False    # Can we currently form sum s?
```

For each number x :

```
for s in range(target, x-1, -1):
    dp[s] = dp[s] or dp[s - x]
```

Why right $\rightarrow$ left (0/1 Knapsack)?

Consider $\text{nums} = [3], \text{target} = 6$.

Initial: T F F F F F F
 0 1 2 3 4 5 6

Going LEFT $\rightarrow$ RIGHT (wrong for 0/1):

$s=3$: $\text{dp}[3] = \text{dp}[0] \rightarrow \text{True}$
 $s=6$: $\text{dp}[6] = \text{dp}[3] \rightarrow \text{True}$ ← WRONG! Used the same 3 twice.

$3 \rightarrow \text{dp}[3]$

↓

used again

↓

$3 + 3 = 6$ ← but there is only one 3!

Going RIGHT $\rightarrow$ LEFT (correct for 0/1):

$s=6$: $\text{dp}[6] = \text{dp}[3] \text{ (old)} \rightarrow \text{False}$

$s=3$: $\text{dp}[3] = \text{dp}[0] \rightarrow \text{True}$

By the time $\text{dp}[3]$ becomes True, we've already passed $s=6$. So $s=6$ sees only the pre-update value of $\text{dp}[3]$.

0/1 Knapsack $\rightarrow$ right-to-left $\rightarrow$ each item used ONCE

Unbounded $\rightarrow$ left-to-right $\rightarrow$ item can propagate forward, reused

One-line rule: 0/1 $\rightarrow$ backwards. Unlimited $\rightarrow$ forwards.

Side-by-Side Array Evolution — $nums = [3]$, $target = 6$

Start: s: 0 1 2 3 4 5 6
 T F F F F F F

===== LEFT → RIGHT (WRONG for 0/1) =====

s=3: dp[3] = dp[3] OR dp[0] dp[0]=T
 T F F [T] F F F
 ▲ ▲
 └─ read ─┘ just written!

s=6: dp[6] = dp[6] OR dp[3] dp[3] is the NEW value
 T F F T F F [T]
 ▲ ▲
 └─ read ─┘
 x same 3 counted twice: 3 + 3 = 6

===== RIGHT → LEFT (CORRECT for 0/1) =====

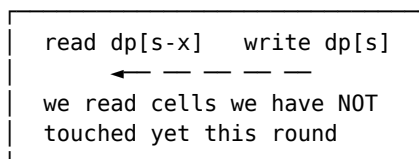
s=6: dp[6] = dp[6] OR dp[3] dp[3] still OLD = F
 T F F F F F [F]
 s=5: dp[5] = dp[5] OR dp[2] = F
 s=4: dp[4] = dp[4] OR dp[1] = F
 s=3: dp[3] = dp[3] OR dp[0] = T
 T F F [T] F F F
 ▲
 written AFTER s=6 was read
 → s=6 never saw it ✓

Final: T F F T F F F
 Only sum 3 is reachable – correct, we have one 3.

The Rule as a Picture

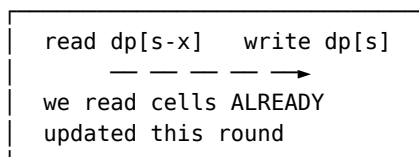
writing direction vs. reading direction

0/1 (right→left):



↓
item used ONCE

Unbounded (left→right):



↓
item can feed itself → reused

This directional trick only matters for the 1D optimization. The original 2D $dp[i][s]$ table needs no such trick.

15. Tree DP

Core idea: Instead of an array index, the state is a **tree node**.

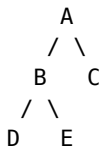
$$dp[\text{node}] = \text{information about the subtree rooted at node}$$

Central question: “What information do I need from my children?”

Computation order: Postorder DFS — children first, then parent. Because the parent’s answer depends on children’s answers, we must compute children first.

15.1 House Robber III (Full Derivation)

Problem:



Choose nodes to maximize total value. Cannot choose two directly connected nodes.

Why one state isn’t enough:

If we define $dp[\text{node}] = \text{best answer for the subtree}$, we lose information. When the parent decides whether to rob B, it needs to know:

- What’s the best if we rob B? (children must be skipped)
- What’s the best if we skip B? (children can be freely chosen)

These are different numbers. So we need **two states per node**.

Define the two states:

$rob[\text{node}] = \text{maximum money if we ROB this node}$
 $skip[\text{node}] = \text{maximum money if we SKIP this node}$

Derivation — Node B with children D, E:

If we ROB B:

- Cannot rob D or E (adjacency rule)
- Must skip both children

$$rob[B] = value[B] + skip[D] + skip[E]$$

If we SKIP B:

- D and E are free — independently choose whichever is better

$\text{skip}[B] = \max(\text{rob}[D], \text{skip}[D]) + \max(\text{rob}[E], \text{skip}[E])$

General recurrence:

$$\text{rob}[\text{node}] = \text{node.val} + \text{skip}[\text{left}] + \text{skip}[\text{right}]$$

$$\text{skip}[\text{node}] = \max(\text{rob}[\text{left}], \text{skip}[\text{left}]) + \max(\text{rob}[\text{right}], \text{skip}[\text{right}])$$

The key insight: The state of the current node determines what choices are available for its children. That's why one state isn't enough.

Leaf nodes (base case):

```
if not node:
    return (0, 0)  # (rob=0, skip=0) - no value, no constraint
```

Computation order:

$D \rightarrow E \rightarrow B \rightarrow C \rightarrow A$ (postorder: children before parents)

Code:

```
class Solution:
    def rob(self, root):
        def dfs(node):
            if not node:
                return (0, 0)

            left_rob, left_skip = dfs(node.left)
            right_rob, right_skip = dfs(node.right)

            # Rob this node: must skip children
            rob = node.val + left_skip + right_skip

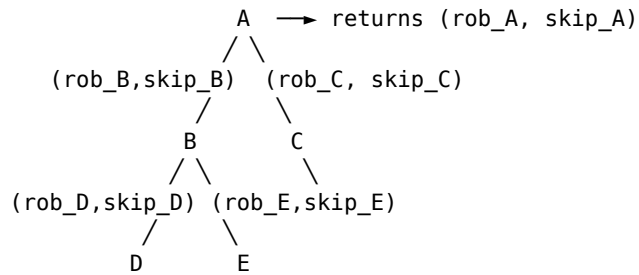
            # Skip this node: children can be either
            skip = max(left_rob, left_skip) + max(right_rob, right_skip)

            return (rob, skip)

        rob_root, skip_root = dfs(root)
        return max(rob_root, skip_root)
```

Recursion Tree = The Actual Tree

In Tree DP the recursion tree *is* the input tree — every node is visited exactly once, and each returns a pair upward:



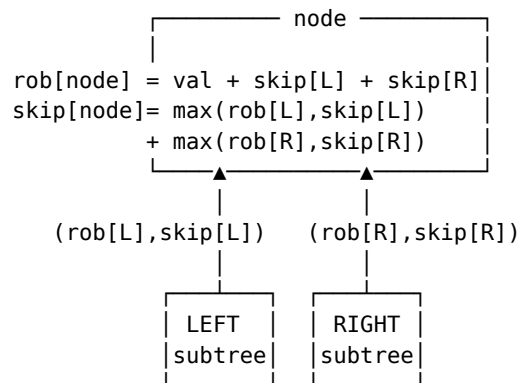
Visit order (postorder): $D \rightarrow E \rightarrow B \rightarrow C \rightarrow A$
 └─ children before parent ─┘

There is NO overlap to memoize here – each subtree is visited once.

What makes it DP is the OPTIMAL SUBSTRUCTURE:

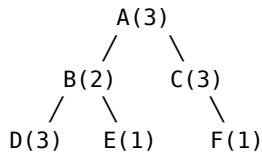
the parent's answer is built from the children's optimal answers.

State Transition Diagram — two states flowing upward



ROB edge : uses ONLY skip from each child (forced)

SKIP edge : uses max(rob, skip) per child (free choice)

Numeric Walkthrough

D (leaf): $\text{rob}[D] = 3 + 0 + 0 = 3$
 $\text{skip}[D] = \max(0,0) + \max(0,0) = 0 \rightarrow (3, 0)$

E (leaf): $\text{rob}[E] = 1 = 1$
 $\text{skip}[E] = 0 = 0 \rightarrow (1, 0)$

F (leaf): $\text{rob}[F] = 1, \text{skip}[F] = 0 \rightarrow (1, 0)$

B: $\text{rob}[B] = 2 + \text{skip}[D] + \text{skip}[E] = 2+0+0 = 2$
 $\text{skip}[B] = \max(3,0) + \max(1,0) = 3+1 = 4 \rightarrow (2, 4)$
 ▲ note: skip beats rob here

C: $\text{rob}[C] = 3 + \text{skip}[F] = 3+0 = 3$
 $\text{skip}[C] = \max(1,0) = 1 \rightarrow (3, 1)$

A: $\text{rob}[A] = 3 + \text{skip}[B] + \text{skip}[C] = 3+4+1 = 8 \leftarrow \text{best}$
 $\text{skip}[A] = \max(2,4) + \max(3,1) = 4+3 = 7 \rightarrow (8, 7)$

Answer = $\max(8, 7) = 8$ (rob A, D, E, F $\rightarrow 3+3+1+1 = 8$)

Notice at node B the *skip* state (4) is larger than the *rob* state (2). If we had stored only a single $\text{dp}[B] = 4$, the parent A could not have computed $\text{rob}[A]$, which needs $\text{skip}[B]$ specifically. **That is exactly why one state is insufficient.**

Final answer: $\max(\text{rob_root}, \text{skip_root})$ — at the root, we choose the better option.

Time: $O(n)$ — each node visited once. **Space:** $O(h)$ recursion stack.

16. Bitmask DP**The Problem First: Travelling Salesman Problem (TSP)****Problem:**

Cities: A B C D

- Start at city A
- Visit every city **exactly once**
- Return to A
- Minimize total travel cost

Example:

$A \rightarrow B = 10, \quad A \rightarrow C = 15, \quad A \rightarrow D = 20$
 $B \rightarrow C = 35, \quad B \rightarrow D = 25, \quad C \rightarrow D = 30$

Route $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A = 10 + 25 + 30 + 15 = 80$

Why Ordinary DP Fails

Suppose we define $dp[\text{current}]$ = minimum cost to reach current. This isn't enough. If we're currently at city D, we need to know:

Which cities have I already visited?

Two different histories ($A \rightarrow B \rightarrow D$ vs. $A \rightarrow C \rightarrow B \rightarrow D$) lead to D, but leave different remaining choices. The history matters.

The Bitmask as a Subset

A bitmask is simply a compact way to represent a **subset of elements**:

A	B	C	D	(bit positions 0, 1, 2, 3)
0	0	0	0	→ no cities visited
1	0	0	0	→ A visited
1	1	0	0	→ A, B visited
1	1	1	0	→ A, B, C visited
1	1	1	1	→ all visited

Operations:

```
# Mark city i as visited:
new_mask = mask | (1 << i)

# Check if city i is visited:
if mask & (1 << i): ... # non-zero means visited
```

TSP DP State

$dp[\text{mask}][\text{current}]$ = minimum cost to reach city `current`,
having visited exactly the cities in `mask`

Transition: try every unvisited city next:

$$dp[\text{mask} \cup \{\text{next}\}][\text{next}] = \min(dp[\text{mask}][\text{current}] + \text{cost}[\text{current}][\text{next}])$$

```
for next_city in range(n):
    if mask & (1 << next_city):
        continue # already visited
    new_mask = mask | (1 << next_city)
    dp[new_mask][next_city] = min(
        dp[new_mask][next_city],
        dp[mask][current] + cost[current][next_city]
    )
```

Base case: Start at city 0 (A), only A is visited, cost so far = 0:

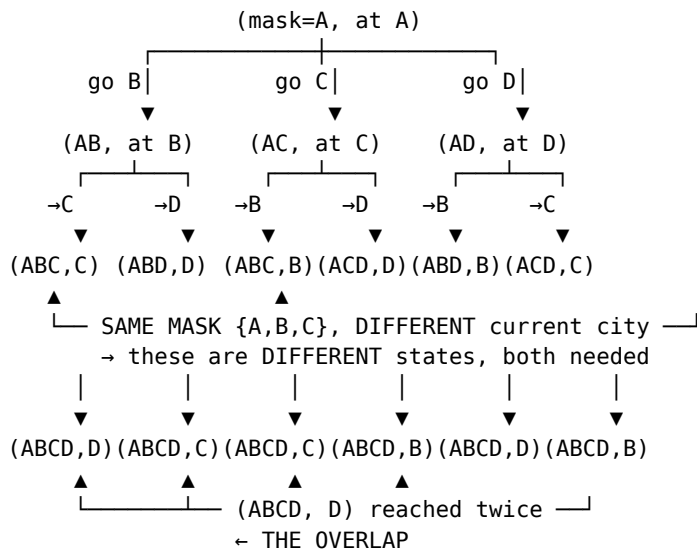
```
dp[1][0] = 0 # mask=0001, current=A(index 0), cost=0
```

Final answer: After visiting all cities (mask = ALL_USED), return to start:

$$\text{answer} = \min_{\text{current}} (dp[\text{ALL_USED}][\text{current}] + \text{cost}[\text{current}][0])$$

Recursion Tree — 4 cities, starting at A

Every level adds one city to the mask. The tree is the set of all permutations — but many nodes collapse:



Permutations: $(n-1)! = 6$ leaves here, 3.6M for $n=11$

States: $2^n \times n = 16 \times 4 = 64$ ← what DP actually stores

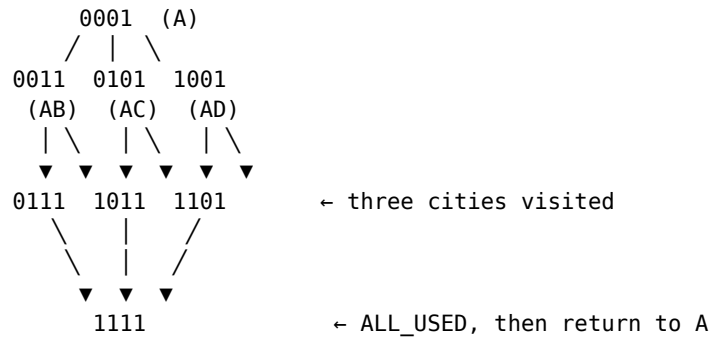
This is the key insight: two different *orders* that visit the same *set* and end at the same *city* are interchangeable from that point on. The bitmask collapses order into set.

```

A → B → C → D      both have mask = 1111
A → C → B → D      both are currently at D
    ↓
the remaining decisions are IDENTICAL
    ↓
merge them → dp[1111][D] = min of the two costs
  
```

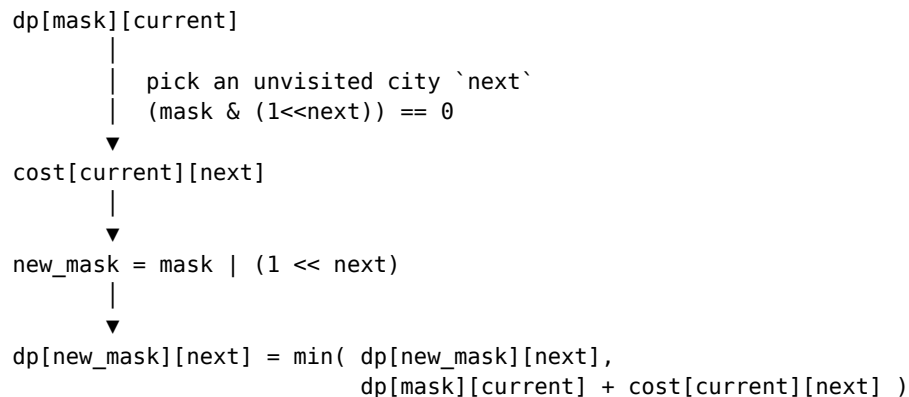
State Graph — how masks grow

mask evolves by turning on one bit at a time:



Layer k of this graph = all masks with k bits set.

Each layer is fully computed before the next → natural DP order.

State Transition Diagram**Worked Trace — the cost table from the problem**

	A	B	C	D
A	-	10	15	20
B	10	-	35	25
C	15	35	-	30
D	20	25	30	-

Start: $dp[0001][A] = 0$

A → B: $dp[0011][B] = 0 + 10 = 10$

A → C: $dp[0101][C] = 0 + 15 = 15$

A → D: $dp[1001][D] = 0 + 20 = 20$

B → D: $dp[1011][D] = 10 + 25 = 35$

D → C: $dp[1111][C] = 35 + 30 = 65$

RETURN TO START: $65 + cost[C][A] = 65 + 15 = 80$

▲
route A→B→D→C→A, total 80

Don't forget the return-to-start cost. The bitmask says all cities are visited, but we still need to travel back to A. Forgetting this final edge is the single most common TSP bug.

Time: $O(2^n \cdot n^2)$ — $2^n \cdot n$ states, each trying n next cities. **Space:** $O(2^n \cdot n)$

Knapsack vs. Bitmask

Need to remember a SUM/CAPACITY?

→ Knapsack DP: `dp[i][target]`

Need to remember WHICH ELEMENTS are used?

→ Bitmask DP: `dp[mask]`

17. Memoization vs. Tabulation

Both implement the same DP logic. They differ in direction.

Memoization (Top-Down)

Start with the original problem

↓

Recursively ask for smaller states

↓

Cache results as they're computed

↓

Return cached result if already computed

```
memo = {}
def dp(state):
    if base_case:
        return base_value
    if state in memo:
        return memo[state]
    ans = recurrence_using_smaller_states
    memo[state] = ans
    return ans

answer = dp(start_state)
```

Advantages: natural recursive thinking; only computes states actually needed.

Tabulation (Bottom-Up)

Start with smallest states (base cases)

↓

Fill in larger states using smaller ones

↓

Iterate in correct order until full answer

```
dp = [initial_value] * n
dp[0] = ...
dp[1] = ...

for i in range(2, n):
    dp[i] = recurrence_using_previous_states

return dp[answer_state]
```

Advantages: no recursion overhead; easier to optimize space.

When to Use Each

	Memoization	Tabulation
Computation	Only needed states	All states
Code style	Recursive	Iterative
Space	Stack + memo dict	DP array
Order	Automatic	Must be explicit
Easier for	Interval DP, Tree DP	Prefix DP, Grid DP

18. Reusable DP Templates

Prefix DP — Memoization

```
def solve(input_data):
    n = len(input_data)
    memo = {}

    def dp(i):
        if i == 0:           # base case
            return base_value
        if i in memo:
            return memo[i]

        # Recurrence: combine smaller states
        ans = ...            # e.g. max(dp(i-1), dp(i-2) + cost)
        memo[i] = ans
        return ans

    return dp(n - 1)
```

Prefix DP — Tabulation

```
def solve(input_data):
    n = len(input_data)
    dp = [0] * n

    # Base cases
    dp[0] = ...
    if n > 1:
        dp[1] = ...

    # Build smaller → larger
    for i in range(2, n):
        dp[i] = ...          # recurrence

    return dp[n - 1]
```

Two-Sequence DP — Memoization

```
def solve(s1, s2):
    n, m = len(s1), len(s2)
    memo = {}

    def dp(i, j):
        if i == 0 or j == 0:
            return base_case
        if (i, j) in memo:
            return memo[(i, j)]

        if condition:          # e.g. s1[i-1] == s2[j-1]
            ans = ...
        else:
            ans = ...

        memo[(i, j)] = ans
        return ans

    return dp(n, m)
```

Two-Sequence DP — Tabulation

```
def solve(s1, s2):
    n, m = len(s1), len(s2)
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    # Base cases
    for i in range(n + 1):
        dp[i][0] = ...
    for j in range(m + 1):
        dp[0][j] = ...

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if condition:      # e.g. s1[i-1] == s2[j-1]
                dp[i][j] = ...
            else:
                dp[i][j] = ...

    return dp[n][m]
```

Shrinking Interval DP — Memoization

```
def solve(s):
    n = len(s)
    memo = {}

    def dp(i, j):
        if i > j:
            return 0          # empty interval
        if i == j:
            return 1          # single element
        if (i, j) in memo:
            return memo[(i, j)]

        if condition:         # e.g. s[i] == s[j]
            ans = ...         # e.g. 2 + dp(i+1, j-1)
        else:
            ans = ...         # e.g. max(dp(i+1, j), dp(i, j-1))

        memo[(i, j)] = ans
        return ans

    return dp(0, n - 1)
```

Shrinking Interval DP — Tabulation

```
def solve(s):
    n = len(s)
    dp = [[0] * n for _ in range(n)]

    # Base: single characters
    for i in range(n):
        dp[i][i] = 1

    # Increasing interval lengths
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if condition:     # e.g. s[i] == s[j]
                dp[i][j] = ...
            else:
                dp[i][j] = ...

    return dp[0][n - 1]
```

Why **length** → **i** → **j** instead of direct **i** → **j**? Smaller intervals ($dp[i+1][j-1]$, $dp[i+1][j]$, $dp[i][j-1]$) must be computed before larger ones. Iterating by length guarantees this.

Partition Interval DP — Memoization

```
def solve(arr):
    n = len(arr)
    memo = {}

    def dp(i, j):
        if i >= j:
            return 0          # base: single element or empty
        if (i, j) in memo:
            return memo[(i, j)]

        ans = float('inf')

        for k in range(i, j): # try every split point
            left = dp(i, k)
            right = dp(k + 1, j)
            cost = ...          # e.g. dims[i-1]*dims[k]*dims[j]
            ans = min(ans, left + right + cost)

        memo[(i, j)] = ans
        return ans

    return dp(0, n - 1)
```

Partition Interval DP — Tabulation

```
def solve(arr):
    n = len(arr)
    dp = [[0] * n for _ in range(n)]

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float('inf')

            for k in range(i, j):
                left = dp[i][k]
                right = dp[k + 1][j]
                cost = ...
                dp[i][j] = min(dp[i][j], left + right + cost)

    return dp[0][n - 1]
```

The length → i → j → k structure is the Partition DP signature. Recognize it immediately.

Tree DP Template

```
def dfs(node):
    if not node:
        return (base_state_1, base_state_2)

    left_state_1, left_state_2 = dfs(node.left)
    right_state_1, right_state_2 = dfs(node.right)

    # Calculate states for current node from children's states
    state_1 = ... # e.g. node.val + left_state_2 + right_state_2
    state_2 = ... # e.g. max(left_state_1, left_state_2) + ...

    return (state_1, state_2)

state_1, state_2 = dfs(root)
answer = max(state_1, state_2) # or whichever combination applies
```

Key points:

- Return **multiple states** when the parent needs different information depending on the current node's choice
- Computation is **postorder** (children first, parent after)
- The number of states depends on the problem; two is common but not fixed

19. Important Distinctions

Two-Sequence DP vs. Interval DP

Both use `dp[i][j]`, but:

Two-Sequence DP:

`i` → position in sequence A (could be different strings/arrays)
`j` → position in sequence B

Interval DP:

`i` → LEFT boundary of a range in ONE sequence
`j` → RIGHT boundary of a range in ONE sequence

Tell them apart: Is `j` an index in a *different* sequence, or the *right end* of the same sequence?

Shrinking Interval vs. Partition Interval

Shrinking:

`[i ..... j]`
 ↓
`[i+1 .... j-1]` (ends shrink inward)
 Question: "What happens to the ends?"

Partition:

`[i ..... j]`
 `k`
`[i ... k][k+1 ... j]` (try every split)
 Question: "Where should I split?"

A problem can belong to both (e.g., Burst Balloons is interval DP and partition DP).

0/1 Knapsack vs. Unbounded Knapsack

0/1 Knapsack:

Each item used at most once.

Recurrence: $dp[i-1][s]$ (always goes to previous row)

1D loop: right $\rightarrow$ left

Unbounded Knapsack:

Each item can be used any number of times.

Recurrence: $dp[i][s-x]$ (stays on same row, same item available again)

1D loop: left $\rightarrow$ right

20. Recognition Cheat Sheet

NEW PROBLEM

↓

What is the subproblem?

↓

ONE SEQUENCE	TWO SEQUENCES	RANGE $[i, j]$
$dp[i]$	$dp[i][j]$	$dp[i][j]$
Prefix DP	Two-Sequence DP	Interval DP
		/ \
		Ends Split
		Shrink Part.

GRID?

$\rightarrow dp[r][c] \rightarrow$ Grid DP $\rightarrow$ "Where did I come from?"

ITEMS + TARGET?

$\rightarrow dp[i][target] \rightarrow$ Knapsack DP $\rightarrow$ "Take or don't take?"

TREE?

$\rightarrow dp[node] \rightarrow$ Tree DP $\rightarrow$ "What do I need from children?"

SUBSET OF ELEMENTS?

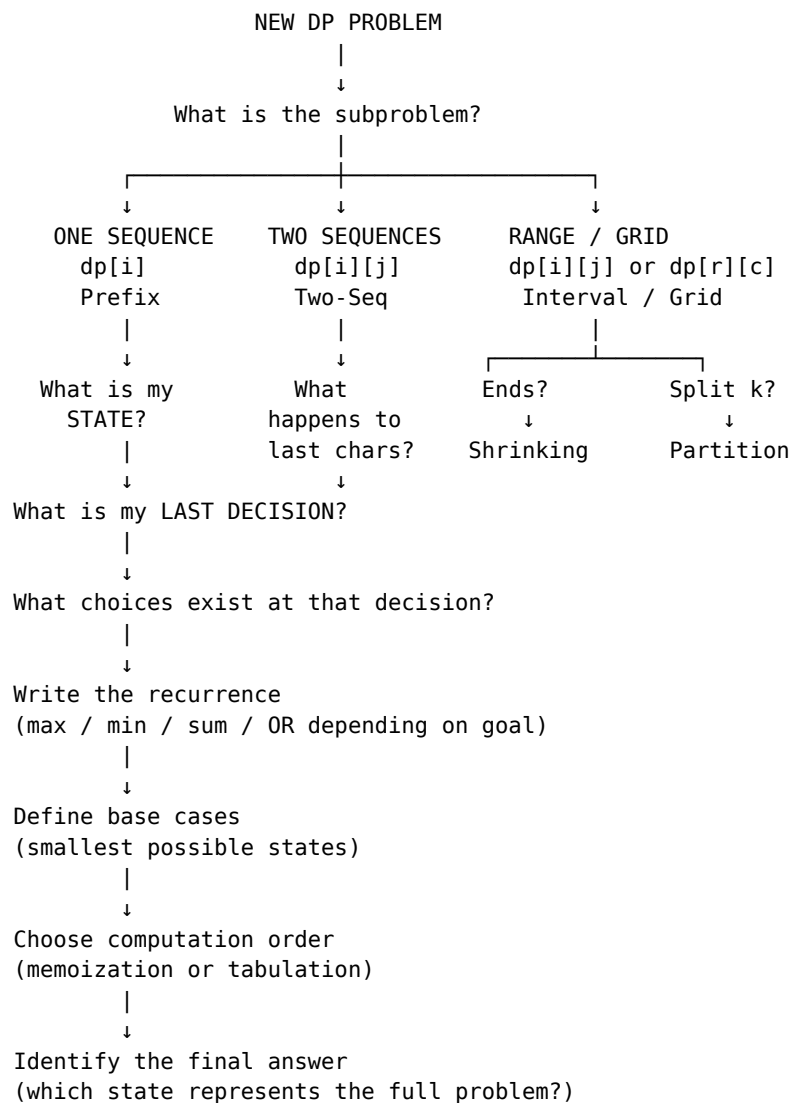
$\rightarrow dp[mask] \rightarrow$ Bitmask DP $\rightarrow$ "What is already used?"

Recognition Table

If the problem mentions...	Think...
first i elements / ending at i	Prefix DP
two strings / two arrays	Two-Sequence DP
substring / subarray / range $[i, j]$	Interval DP
split / partition / cut / parenthesize	Partition DP
grid / matrix / board / paths	Grid DP

If the problem mentions...	Think...
subset / target / capacity / take-or-skip	Knapsack DP
tree / subtree / children / parent	Tree DP
visited subset / used elements / permutations	Bitmask DP

21. The DP Thinking Procedure — One Page



22. Common Mistakes

These mistakes were encountered during the learning session:

State definition mistakes:

- Confusing the *state* with the *final answer*. The state is a general subproblem; the answer is one specific state.
- Choosing a state that lacks necessary information (e.g., one `dp[node]` for Tree DP when two states are needed).
- Assuming `dp[i]` when the problem involves two sequences — the state should be `dp[i][j]`.

Recurrence mistakes:

- Forgetting the +1 in Coin Change: `dp[x] = min(dp[x-c])` is wrong; it should be `min(dp[x-c] + 1)`.
- Using an overly specific condition in LIS: `nums[j] == nums[i] - 1` is wrong; the correct condition is `nums[j] < nums[i]`.
- In Distinct Subsequences, using only `dp[i-1][j-1]` for the matching case — the skip option `dp[i-1][j]` must also be included.
- In Edit Distance, swapping insert and delete (which DP state corresponds to which operation).

Final answer mistakes:

- Assuming `dp[n-1]` is always the answer. For LIS, the answer is `max(dp[i])` over all `i`.
- In TSP, forgetting the final return-to-start cost even after all cities are visited.

Pattern confusion:

- Confusing Two-Sequence DP with Interval DP — both use `dp[i][j]`. Ask: are `i` and `j` in different sequences, or the same?
- Confusing Shrinking Interval with Partition — “look at the ends” vs. “try every split point `k`”.

Knapsack mistakes:

- Accidentally reusing an item in 0/1 Knapsack by going left→right in the 1D optimization. The `nums=[3]`, `target=6` example: left→right incorrectly gives `dp[6]=True` using the same 3 twice.

Tree DP mistakes:

- Introducing Bitmask DP before understanding the underlying problem (TSP). Always understand the problem first.
- Not computing Tree DP in postorder (children must be computed before their parent).

Grid DP mistakes:

- Using `min` for Unique Paths (should be `+`) or `+` for Minimum Path Sum (should be `min`). Ask: am I counting paths, or minimizing cost?
- In Maximal Square, forgetting the +1 in `1 + min(top, left, diagonal)`.

This material was derived from a complete live learning session on DP patterns. The reasoning, derivations, analogies, and corrections are sourced directly from that session.